HPC Project 3: Random Matrics

Addison Olstad

April 17, 2026

Contents

1 Introduction **2**

2 Simulation **4**

2.1 Manager-Worker Architecture 4

2.2 Numerical Methods 5

2.2.1 Power Method 5

2.2.2 Schur Decomposition 6

2.3 Wall Time 7

3 Results **8**

3.0.1 Walltime 8

3.0.2 Schur Usage 8

3.0.3 Distributions 9

3.0.4 Statistical Tests 11

Chapter 1

Introduction

Computing the largest eigenvalue of a single random matrix is not a particularly demanding task. The problem becomes computationally difficult, however, when one must generate and analyze thousands of large matrices. At scale, the memory requirements alone present a significant constraint: storing M matrices of size $n \times n$ in double precision requires $M \cdot n^2 \cdot 8$ bytes of RAM, which swiftly exceeds what a typical personal computer can hold. Beyond memory, there are several compounding bottlenecks: filling each matrix with random entries, computing its eigenvalues, selecting the maximum, and writing the results to disk. Doing this sequentially on a single processor is impractical for any statistically meaningful sample size.

The matrices considered here are drawn from the Gaussian Orthogonal Ensemble (GOE). Each matrix A is real and symmetric, with off-diagonal entries A_{ij} drawn independently from $\mathcal{N}(0, \frac{1}{2})$ and diagonal entries from $\mathcal{N}(0, 1)$, with symmetry enforced by setting $A_{ji} = A_{ij}$. For an ensemble of M such $n \times n$ matrices, the output of interest is the largest eigenvalue $\lambda_{\max}$ of each realization, and in particular the distribution this quantity follows across the ensemble.

To make this computation feasible, the problem is parallelized using MPI with a manager-worker layout. Because each matrix realization is statistically independent, the problem is "embarrassingly parallel"; the manager distributes work orders to workers, each of which generates a single matrix, computes its largest eigenvalue, and returns the result. The largest eigenvalue is computed using the Power Method, an iterative algorithm that efficiently approximates the dominant eigenvalue, which runs in $\mathcal{O}(n^2)$ and is sufficient for most realizations. In cases where the Power Method fails to converge, a Schur decomposition via LAPACK serves as a robust $\mathcal{O}(n^3)$ fallback, as it computes the full eigenvalue spectrum.

Algorithm 1 Maximum eigenvalue ensemble

Require: Number of realizations M, matrix size n
Ensure: $\{\lambda_{\max}^{(s)}\}_{s=1}^{M}$

```
for s = 1 to M do
    Construct A^(s) ∈ R^(n×n) using a PRNG
    Compute eigenvalues of A^(s), select λ_max^(s)
end for
```

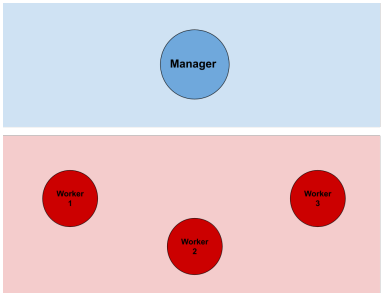
With the ensemble of eigenvalues collected, two hypotheses are tested. The first is that the distribution of $\lambda_{\max}$ converges to a Gaussian distribution in the limit $n \to \infty$. The second is that it instead converges to the Tracy–Widom distribution predicted by random matrix theory. The Tracy–Widom

distribution is the limiting distribution of the scaled largest eigenvalue of a random matrix drawn from Gaussian ensembles [2]. These hypotheses will be evaluated using goodness-of-fit tests such as the Shapiro–Wilk and Kolmogorov–Smirnov tests, along with moment-based measures (skewness and excess kurtosis), to quantify deviations from normality.

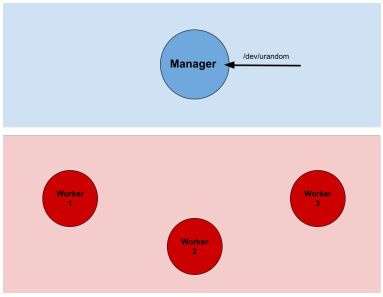
Chapter 2

Simulation

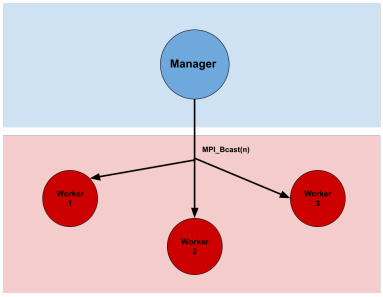
2.1 Manager-Worker Architecture



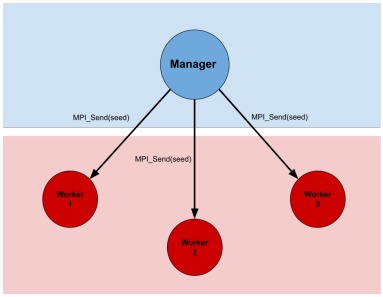
(a) Initialization



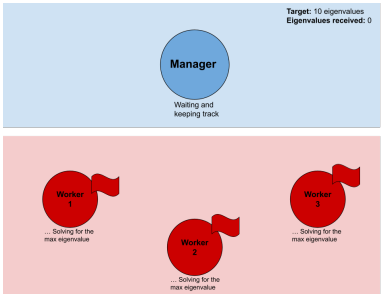
(b) Reading the Seed



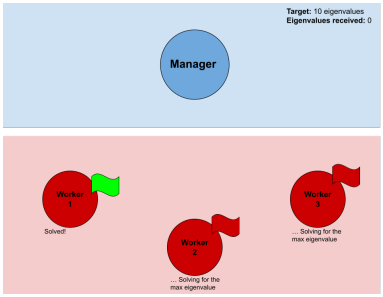
(c) Broadcasting the Seeds



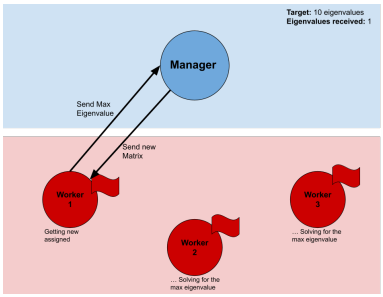
(d) Dispatching Work



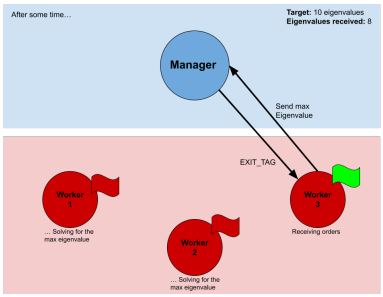
(e) Workers Computing



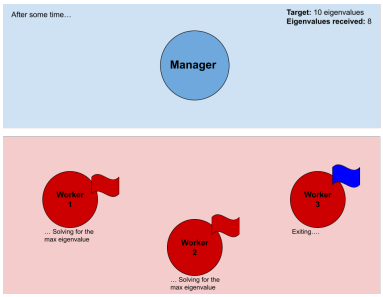
(f) First Result In



(g) Dynamic Redispatch



(h) Exit Tag Sent



(i) Winding Down

Figure 2.1: The Manager Worker Architecture Flow Chart

(add a written version of the manager worker structure within the code, maybe some pseudocode (or

maybe not))

2.2 Numerical Methods

A naive approach to computing eigenvalues is via the characteristic polynomial:

$$P_A(x) = \det(xI - A) \tag{2.1}$$

For sizable matrices this is not feasible because computing it is intensive, numerically unstable, and theoretically difficult. Instead numerical iterative methods can find eigenvalues directly with higher efficiency. In this project two methods were explored within the program: the Power method (Von Mises Iteration) and the Schur decomposition.

2.2.1 Power Method

The Power method, also known as Von Mises iteration, computes the eigenvalue of a diagonalizable matrix A that is greatest in absolute value, along with its corresponding eigenvector. Each iteration costs $\mathcal{O}(n^2)$, dominated by a single matrix–vector multiplication. The method requires that A possess a *strictly dominant* eigenvalue, one that is strictly larger in absolute value than all others, and that the initial vector $\mathbf{b}_0$ have a nonzero component in the direction of the dominant eigenvector. If either condition fails, convergence is not guaranteed.

Starting from a random initial vector $\mathbf{b}_0$, each iteration multiplies by A and renormalizes:

$$\mathbf{b}_{k+1} = \frac{A\mathbf{b}_k}{\|A\mathbf{b}_k\|}.$$

The sequence $\{\mathbf{b}_k\}$ converges to the dominant eigenvector. At each step the corresponding eigenvalue is estimated via the Rayleigh quotient,

$$\lambda \approx \mathbf{b}_k^\top A\,\mathbf{b}_k,$$

and iteration continues until the relative change in λ falls below a prescribed tolerance ε:

$$\frac{|\lambda_{\text{new}} - \lambda_{\text{old}}|}{|\lambda_{\text{old}}|} < \varepsilon.$$

Eigenvalues of a large GOE matrix are distributed about zero, so the dominant eigenvalue returned by the basic algorithm may be negative. To recover the true spectral maximum, a spectral shift μ is applied: the power method is run on the shifted matrix $(A + \mu I)$, whose eigenvalues are those of A displaced by μ. On the first call $\mu = 0$; if the result is negative the method is restarted with $\mu = -\lambda$, which flips the spectrum and ensures the largest positive eigenvalue is dominant.

The subroutine `power(A, mu, eps, M, eig, conv)` accepts the matrix A, shift μ, tolerance `eps`, and maximum iteration count `M`, and returns the estimated dominant eigenvalue `eig` together with a convergence flag `conv`. Setup initializes a random starting vector drawn from $\mathcal{N}(0, 1)$ using Intel MKL's `vdrnggaussian`, which is then normalized to unit length. The value `mem`$= -100$ is used to avoid a division-by-zero in the relative-change check on the first iteration.

The $\mathcal{O}(n^2)$ BLAS routine `dgemv` performs the matrix–vector product $y \leftarrow Av$ and is the computational bottleneck of each iteration. The shift is applied additively after the multiply ($y \mathrel{+}= \mu v$), and the Rayleigh quotient is unshifted before storage ($\lambda \mathrel{-}= \mu$) so that `eig` always reports the eigenvalue of the original matrix A.

Algorithm 2 Power Method Subroutine

```
conv ← false
Generate Gaussian random vector v
Normalize v ← v/‖v‖
mem ← −100
for i = 1 to M do
    y ← Av
    y ← y + μv
    nrm ← ‖y‖
    eig ← v^T y
    eig ← eig − μ
    err ← |mem − eig| / |mem|
    if err < ε then
        conv ← true
        break
    end if
    v ← y/nrm
    mem ← eig
end for
```

2.2.2 Schur Decomposition

When the power method fails to converge, for example when the dominant eigenvalue assumption is violated, the corresponding realization is not discarded. Instead, a fallback Schur decomposition is used to compute the eigenvalue exactly. In this way, every matrix contributes one eigenvalue to the ensemble, so the resulting distribution is not biased by selectively rejecting computationally difficult cases.

The Schur decomposition states that any complex square matrix A is unitarily similar to an upper triangular matrix T,

$$A = QTQ^*,$$

where the diagonal entries of T are precisely the eigenvalues of A. In practice this decomposition is computed via the QR algorithm. Because it produces the full spectrum rather than a single extrema eigenvalue, it is considerably more expensive than the power method at $\mathcal{O}(n^3)$, but it is robust and requires no assumptions about spectral dominance.

The fallback is programmed immediately after the power method returns with `conv = .false.`, using LAPACK's `DGEEV` routine to compute all eigenvalues of A:

To record which method produced each result, the eigenvalue is sent to the root process via MPI with a tag encoding the convergence path: tag 0 indicates the power method converged, while tag 1 indicates the Schur fallback was used.

These tags are collected by the root process alongside the eigenvalue data, allowing the results section to report separately on the fraction of realizations that required the fallback.

Algorithm 3 Fallback to DGEEV

```
if ¬conv then
    DGEEV(...)
    if info = 0 then
        eig ← max(wr)
        conv ← true
    else
        print "error in DGEEV", info
    end if
end if
```

Algorithm 4 MPI Send Result

```
if conv then
    tag ← 0
else
    tag ← 1
end if
MPI_SEND(eig, 1, MPI_DOUBLE_PRECISION, 0, tag, MPI_COMM_WORLD, ierr)
```

2.3 Wall Time

Wall time measures the total elapsed time from the start of program execution to completion. In this simulation, the computational cost of each realization is dominated by the eigenvalue computation. The power method requires $\mathcal{O}(n^2)$ operations per iteration, while the fallback Schur decomposition requires $\mathcal{O}(n^3)$ operations. Since the fallback is only called when the power method fails to converge, the overall cost per realization lies between these two bounds, depending on the frequency of fallback usage.

Under MPI parallel execution with p workers, the ensemble is distributed across processes, ideally reducing the wall time according to

$$T_{\text{wall}} \approx \frac{T_{\text{serial}}}{p}.$$

However, in practice, deviations from ideal scaling arise due to MPI communication overhead, variability in convergence rates of the power method, and occasional use of the more expensive Schur fallback. These factors introduce load imbalance across workers, leading to reduced parallel efficiency for large p or large matrix sizes.

Chapter 3

Results

(insert some brief introduction to the results)

3.0.1 Walltime

- running on sharknet - n values of 1000, 2000, 4000, 6000 for 1000 iterations each - recieved these walltime numbers - graphed between the spectral gap of $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$

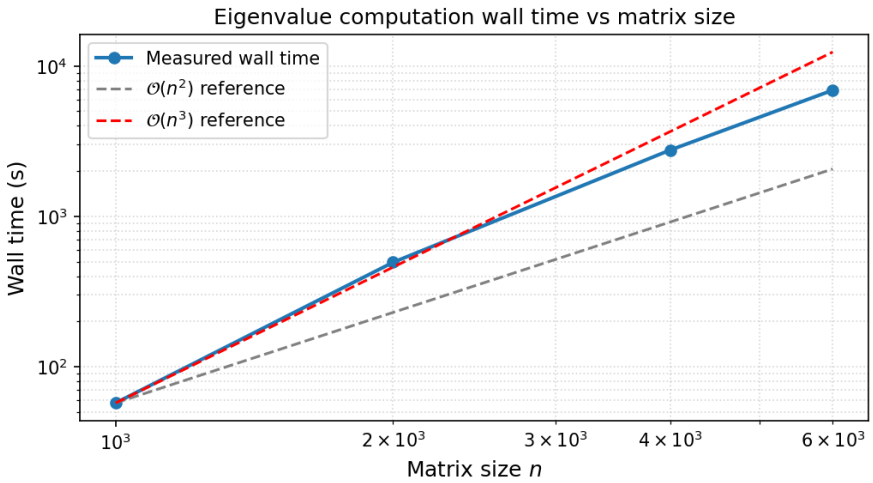


Figure 3.1: Walltime as a function of matrix size n.

Expected results

3.0.2 Schur Usage

Explain process of collecting tags, writing to file, collecting when power vs schur was used. Then producing a percentage rate.

This was for the largest matrix size computed, alongside N=1000,2000, and 4000. Those matrices had the same results as the 6000 sized matrix.

- slightly unexpected - brief hypothesis about power method's convergence assumptions

Quantity	Value
Total eigenvalues collected	1000
Power method converged	1000
Schur fallback used	0
Schur fallback rate	0.00%

Table 3.1: Eigenvalue computation summary for $n = 6000$.

3.0.3 Distributions

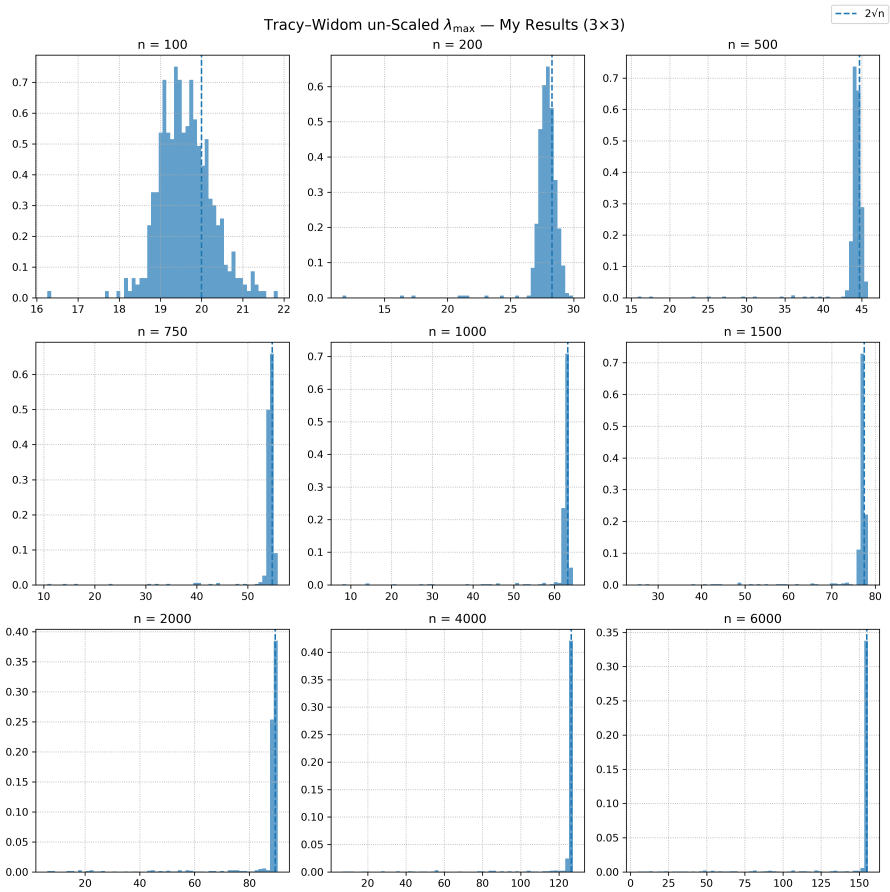


Figure 3.2: Empirical distribution of $\lambda_{\max}$ without tolerance

- unexpected...

To ensure the observed distributions reflect the true statistical behavior of the largest eigenvalue, a step was applied before visualization in the histogram. The raw data includes extreme left-tail values that are inconsistent with the theoretical spectral edge $2\sqrt{n}$ of GOE matrices. (insert theory about why? or save for later?)

- filtering based on median and IQR - remove extreme left-tail artifcats without affecting bulk - then shift largest eigenvalues by $2\sqrt{n}$ and rescaled by $n^{-1/6}$ [1], the known fluctuation scale from random matrix theory.

Algorithm 5 Tracy–Widom Scaling and Cleaning Procedure

Require: Eigenvalue samples $\{\lambda_i\}$, matrix size n
Ensure: Cleaned and normalized samples $\{z_i\}$

```
Compute spectral edge: t_w ← 2√n
Compute median: m ← median({λ_i})
Compute interquartile range: IQR ← Q_75 − Q_25
Filter numerical artifacts:
{λ_i^clean} ← {λ_i | λ_i > m − 5 · IQR}
Apply Tracy–Widom scaling:
z_i ← (λ_i^clean − t_w) / n^(1/6)
Remove extreme scaled values:
{z_i^clean} ← {z_i | −6 < z_i < 4}
return {z_i^clean}
```

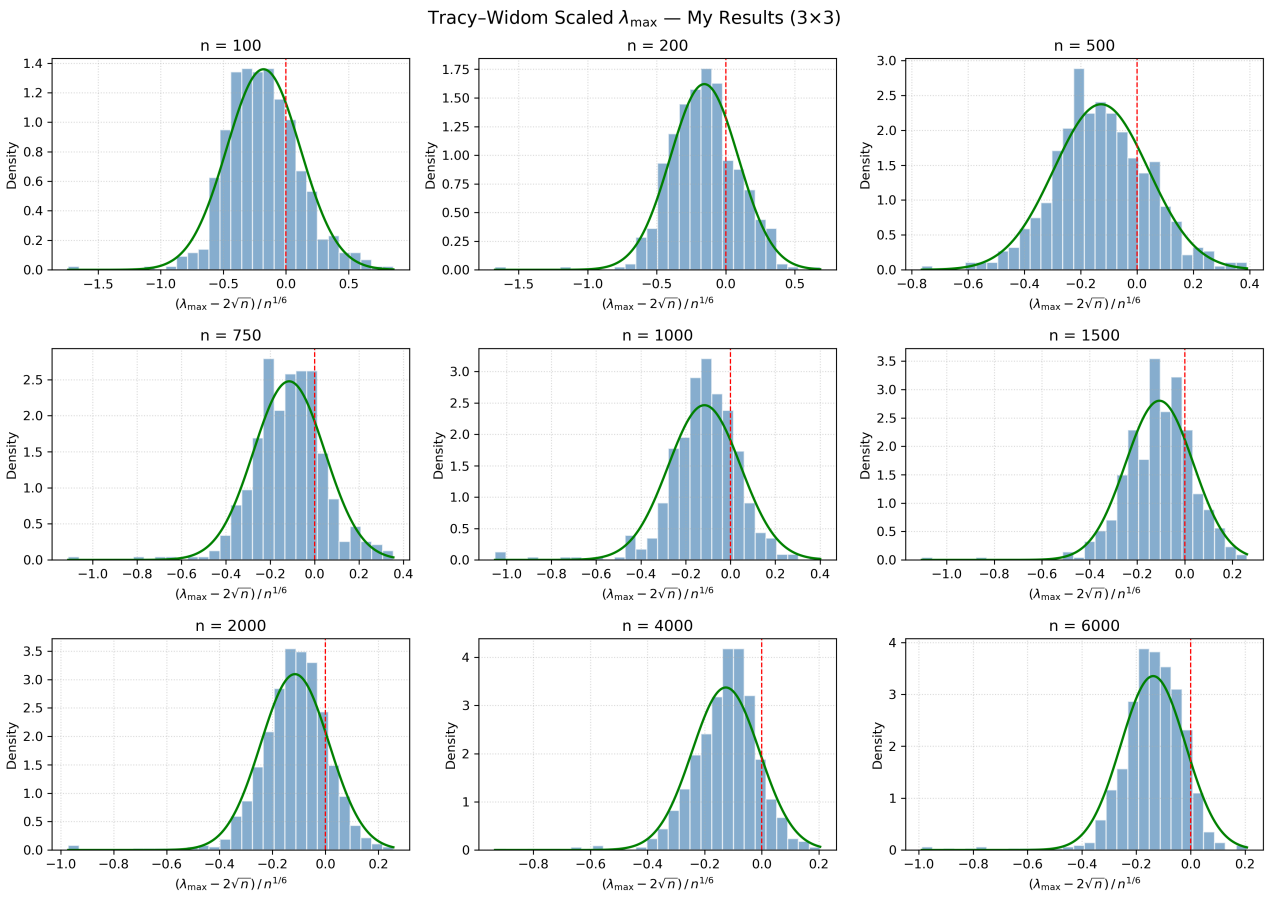


Figure 3.3: Empirical distribution of $\lambda_{\max}$ with normal overlay.

This output, with the integrated tolerance demonstrates a more expected result.

These additional plots arise from another computation completed with the n values of 1024, 2048, 4096, and 8192

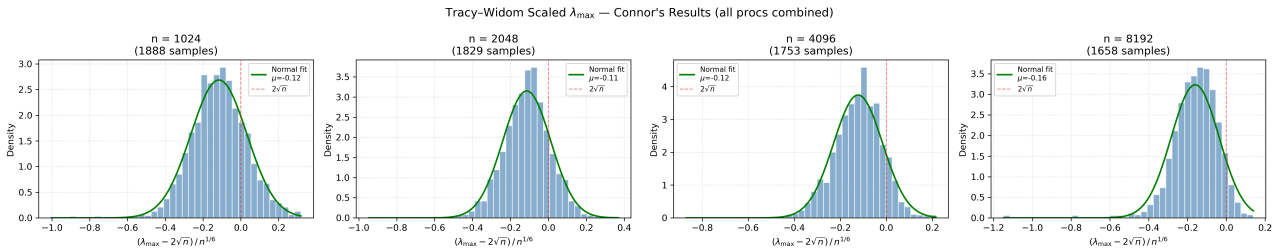


Figure 3.4: Cleaned $\lambda_{\max}$ distribution with theoretical overlay.

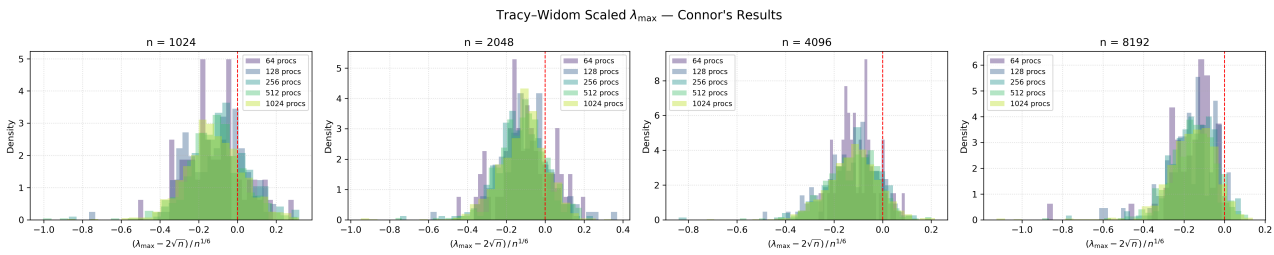


Figure 3.5: Raw $\lambda_{\max}$ distribution.

These plots demonstrate the n values at different procs levels. Addy note: totally forgot to define the red line in the legend.

3.0.4 Statistical Tests

loading... brief intro into shapiro–wilk, kolmogorov-smirnov, and moment-based mesaures

Shapiro–Wilk

loading...

Kolmogorov–Smirnov

loading...

Moment-Based Measures

loading...

Bibliography

[1] Wikipedia contributors. Tracy–widom distribution. https://en.wikipedia.org/wiki/Tracy%E2%80%93Widom_distribution, 2026. Accessed: 2026-04-17.

[2] Wolfram Research. Tracy–widom distribution. https://www.wolfram.com/language/11/random-matrices/tracy-widom-distribution.html, 2015. Accessed: 2026-04-15.